

Chapter Framework: Ch.4 Numbers + Ch.5 Conditionals + Ch.6 Functions

 **Combination Challenge** — This task requires skills from input validation (Ch.6), conditional pricing (Ch.5), and arithmetic (Ch.4). Apply each skill where appropriate.

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates the delivery fee for an online shop based on the parcel weight and the chosen delivery priority. The function must validate inputs before calculating.

Delivery Pricing Rules:

- ▶ If weight is not a number (int or float) or is less than or equal to 0, return: Invalid weight
- ▶ If priority is not 'standard' or 'express', return: Unsupported priority
- ▶ Priority 'standard': Base cost \$8. If weight is over 3 kg, add \$2 for every kg over 3.
- ▶ Priority 'express': Base cost \$15. If weight is over 3 kg, a surcharge of 20% of the base cost is added.
- ▶ Return the final cost rounded to 2 decimal places.

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named calculate_delivery_fee.
2. The function should accept two parameters: weight (in kg) and priority.
3. If weight is not a number (int or float) or is <= 0, return: Invalid weight.
4. If priority is not 'standard' or 'express', return: Unsupported priority.
5. For 'standard': base cost is \$8; add \$2 per kg for every kg over 3.
6. For 'express': base cost is \$15; if over 3 kg, add a 20% surcharge on the \$15 base.
7. Return the final cost rounded to 2 decimal places.

Tests (8 marks)
1. You should have a function named calculate_delivery_fee.
2. calculate_delivery_fee(-1, 'standard') → should return "Invalid weight".
3. calculate_delivery_fee(2, 'vip') → should return "Unsupported priority".

4. calculate_delivery_fee(2, 'standard') → should return 8.0.
5. calculate_delivery_fee(5, 'standard') → should return 12.0.
6. calculate_delivery_fee(2, 'express') → should return 15.0.
7. calculate_delivery_fee(5, 'express') → should return 18.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: [YourClass][YourNumber].py (e.g. S3D05.py)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Invalid weight
Unsupported priority
8.0
12.0
15.0
18.0
```

— — — **End of Paper** — — —

Chapter Framework: Ch.4 Numbers + Ch.5 Conditionals + Ch.6 Functions

⚡ **Combination Challenge** — This task requires skills from input validation (Ch.6), conditional discounts (Ch.5), and multiplication (Ch.4). Apply each skill where appropriate.

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates the total gym membership fee based on the number of months and the chosen membership plan. Long-term commitments earn a discount.

Gym Membership Pricing Rules:

- ▶ If months is not an integer or is less than or equal to 0, return: Invalid months
- ▶ If plan is not 'basic' or 'premium', return: Invalid plan
- ▶ Plan 'basic': Monthly rate is \$80. If months >= 6, apply a 10% discount to the total.
- ▶ Plan 'premium': Monthly rate is \$120. If months >= 12, apply a 15% discount to the total.
- ▶ Return the final fee rounded to 2 decimal places.

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named calculate_gym_fee.
2. The function should accept two parameters: months (whole number) and plan.
3. If months is not an integer or is <= 0, return: Invalid months.
4. If plan is not 'basic' or 'premium', return: Invalid plan.
5. For 'basic': charge \$80 per month; if months >= 6, deduct 10% from the total.
6. For 'premium': charge \$120 per month; if months >= 12, deduct 15% from the total.
7. Return the final fee rounded to 2 decimal places.

Tests (8 marks)
1. You should have a function named calculate_gym_fee.
2. calculate_gym_fee(-1, 'basic') → should return "Invalid months".
3. calculate_gym_fee(3, 'gold') → should return "Invalid plan".
4. calculate_gym_fee(3, 'basic') → should return 240.0.

5. calculate_gym_fee(6, 'basic') → should return 432.0.
6. calculate_gym_fee(6, 'premium') → should return 720.0.
7. calculate_gym_fee(12, 'premium') → should return 1224.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① **A Python file named:** `[YourClass][YourNumber].py` (e.g. `S3D05.py`)
- ② **A screenshot** showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Invalid months
Invalid plan
240.0
432.0
720.0
1224.0
```

— — — **End of Paper** — — —

Chapter Framework: Ch.4 Numbers + Ch.5 Conditionals + Ch.6 Functions

 **Combination Challenge** — This task requires skills from input validation (Ch.6), time-based pricing (Ch.5), and per-km arithmetic (Ch.4). Apply each skill where appropriate.

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates the taxi fare based on the distance travelled and the time of day. Night trips include a flat surcharge, and longer journeys incur a per-kilometre excess charge.

Taxi Fare Pricing Rules:

- ▶ If distance is not a number (int or float) or is less than or equal to 0, return: Invalid distance
- ▶ If time_of_day is not 'day' or 'night', return: Unsupported time of day
- ▶ Time 'day': Base fare \$20. If distance > 10 km, add \$1.50 for every km over 10.
- ▶ Time 'night': Base fare \$30 plus a flat \$5 night surcharge. If distance > 10 km, add \$2.00 for every km over 10.
- ▶ Return the final fare rounded to 2 decimal places.

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named calculate_taxi_fare.
2. The function should accept two parameters: distance (in km) and time_of_day.
3. If distance is not a number (int or float) or is <= 0, return: Invalid distance.
4. If time_of_day is not 'day' or 'night', return: Unsupported time of day.
5. For 'day': base fare is \$20; add \$1.50 per km for every km over 10.
6. For 'night': base fare is \$30 + \$5 surcharge; add \$2.00 per km for every km over 10.
7. Return the final fare rounded to 2 decimal places.

Tests (8 marks)
1. You should have a function named calculate_taxi_fare.
2. calculate_taxi_fare(-5, 'day') → should return "Invalid distance".
3. calculate_taxi_fare(8, 'morning') → should return "Unsupported time of day".

4. calculate_taxi_fare(8, 'day') → should return 20.0.
5. calculate_taxi_fare(15, 'day') → should return 27.5.
6. calculate_taxi_fare(8, 'night') → should return 35.0.
7. calculate_taxi_fare(15, 'night') → should return 45.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: [YourClass][YourNumber].py (e.g. S3D05.py)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Invalid distance
Unsupported time of day
20.0
27.5
35.0
45.0
```

— — — **End of Paper** — — —

Chapter Framework: Ch.4 Numbers + Ch.5 Conditionals + Ch.6 Functions

⚡ Combination Challenge — This task requires skills from input validation (Ch.6), vehicle-based pricing (Ch.5), and discount arithmetic (Ch.4). Apply each skill where appropriate.

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates the total car rental cost based on the number of days and the type of vehicle selected. Long rentals receive a discount (economy) or incur an insurance surcharge (luxury).

Car Rental Pricing Rules:

- ▶ If days is not an integer or is less than or equal to 0, return: Invalid days
- ▶ If car_type is not 'economy' or 'luxury', return: Unsupported car type
- ▶ Type 'economy': Daily rate is \$50. If rental is more than 7 days, a \$5 per-day discount is applied to all days.
- ▶ Type 'luxury': Daily rate is \$120. If rental is more than 7 days, a 10% insurance surcharge is added to the total.
- ▶ Return the final cost rounded to 2 decimal places.

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named calculate_rental_cost.
2. The function should accept two parameters: days (whole number) and car_type.
3. If days is not an integer or is <= 0, return: Invalid days.
4. If car_type is not 'economy' or 'luxury', return: Unsupported car type.
5. For 'economy': daily rate \$50; if days > 7, reduce rate to \$45 for all days.
6. For 'luxury': daily rate \$120; if days > 7, add 10% insurance surcharge to the total.
7. Return the final cost rounded to 2 decimal places.

Tests (8 marks)
1. You should have a function named calculate_rental_cost.
2. calculate_rental_cost(-2, 'economy') → should return "Invalid days".

3. calculate_rental_cost(3, 'sedan') → should return "Unsupported car type".
4. calculate_rental_cost(3, 'economy') → should return 150.0.
5. calculate_rental_cost(10, 'economy') → should return 450.0.
6. calculate_rental_cost(3, 'luxury') → should return 360.0.
7. calculate_rental_cost(10, 'luxury') → should return 1320.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: `[YourClass][YourNumber].py` (e.g. `S3D05.py`)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Invalid days
Unsupported car type
150.0
450.0
360.0
1320.0
```

— — — **End of Paper** — — —

Chapter Framework: Ch.4 Numbers + Ch.5 Conditionals + Ch.6 Functions

⚡ Combination Challenge — This task requires skills from input validation (Ch.6), service-based pricing (Ch.5), and tiered-rate arithmetic (Ch.4). Apply each skill where appropriate.

Please use **Thonny** to complete the task. Add these lines at the top of your code:

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")
```

Instructions

Write a function that calculates the repair fee for an electronics shop based on the number of hours worked and the type of service required. Extra hours are discounted for basic service; specialist jobs incur an urgent fee.

Repair Pricing Rules:

- ▶ If hours is not a number (int or float) or is less than or equal to 0, return: Invalid hours
- ▶ If service_type is not 'basic' or 'specialist', return: Unsupported service type
- ▶ Service 'basic': Hourly rate is \$60. If hours > 2, extra hours are charged at 80% of the normal rate.
- ▶ Service 'specialist': Hourly rate is \$100. If hours > 2, a flat \$30 urgent fee is also charged.
- ▶ Return the final fee rounded to 2 decimal places.

Objective: Fulfill the user stories below to complete the task.

User Stories
1. You should define a function named calculate_repair_fee.
2. The function should accept two parameters: hours (number of hours) and service_type.
3. If hours is not a number (int or float) or is <= 0, return: Invalid hours.
4. If service_type is not 'basic' or 'specialist', return: Unsupported service type.
5. For 'basic': charge \$60/hr for first 2 hrs; extra hours charged at \$60 × 80%.
6. For 'specialist': charge \$100/hr; if hours > 2, also add a flat \$30 urgent fee.
7. Return the final fee rounded to 2 decimal places.

Tests (8 marks)
1. You should have a function named calculate_repair_fee.
2. calculate_repair_fee(0, 'basic') → should return "Invalid hours".

3. calculate_repair_fee(1, 'expert') → should return "Unsupported service type".
4. calculate_repair_fee(1.5, 'basic') → should return 90.0.
5. calculate_repair_fee(3, 'basic') → should return 168.0.
6. calculate_repair_fee(2, 'specialist') → should return 200.0.
7. calculate_repair_fee(4, 'specialist') → should return 430.0.

Submission (2 marks)

Please submit the following two files to Google Classroom:

- ① A Python file named: [YourClass][YourNumber].py (e.g. S3D05.py)
- ② A screenshot showing your complete code and output in the *Thonny IDE*.

Sample Output

```
Invalid hours
Unsupported service type
90.0
168.0
200.0
430.0
```

— — — **End of Paper** — — —

ANSWER KEY

Model answers for all 5 practice papers. Accept any logically equivalent solution.

Paper A — Build a Delivery Fee Calculator

Skills: `isinstance()` · `if/elif/else` · `arithmetic` · `round()`

Function: `calculate_delivery_fee(weight, priority)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_delivery_fee(weight, priority):
    if not isinstance(weight, (int, float)) or weight <= 0:
        return "Invalid weight"
    if priority not in ("standard", "express"):
        return "Unsupported priority"
    if priority == "standard":
        cost = 8
        if weight > 3:
            cost += 2 * (weight - 3)
    else:
        cost = 15
        if weight > 3:
            cost += 15 * 0.20
    return round(cost, 2)

print(calculate_delivery_fee(-1, 'standard'))
print(calculate_delivery_fee(2, 'vip'))
print(calculate_delivery_fee(2, 'standard'))
print(calculate_delivery_fee(5, 'standard'))
print(calculate_delivery_fee(2, 'express'))
print(calculate_delivery_fee(5, 'express'))
```

Paper B — Build a Gym Membership Fee Calculator

Skills: `isinstance()` · `if/elif/else` · `arithmetic` · `round()`

Function: `calculate_gym_fee(months, plan)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_gym_fee(months, plan):
    if not isinstance(months, int) or months <= 0:
        return "Invalid months"
    if plan not in ("basic", "premium"):
        return "Invalid plan"
    if plan == "basic":
        total = 80 * months
        if months >= 6:
            total *= 0.90
    else:
        total = 120 * months
        if months >= 12:
            total *= 0.85
    return round(total, 2)

print(calculate_gym_fee(-1, 'basic'))
print(calculate_gym_fee(3, 'gold'))
print(calculate_gym_fee(3, 'basic'))
```



```
print(calculate_gym_fee(6, 'basic'))
print(calculate_gym_fee(6, 'premium'))
print(calculate_gym_fee(12, 'premium'))
```

Paper C — Build a Taxi Fare Calculator

Skills: *isinstance()* · *if/elif/else* · *arithmetic* · *round()*

Function: `calculate_taxi_fare(distance, time_of_day)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_taxi_fare(distance, time_of_day):
    if not isinstance(distance, (int, float)) or distance <= 0:
        return "Invalid distance"
    if time_of_day not in ("day", "night"):
        return "Unsupported time of day"
    if time_of_day == "day":
        fare = 20
        if distance > 10:
            fare += 1.5 * (distance - 10)
    else:
        fare = 30 + 5
        if distance > 10:
            fare += 2.0 * (distance - 10)
    return round(fare, 2)

print(calculate_taxi_fare(-5, 'day'))
print(calculate_taxi_fare(8, 'morning'))
print(calculate_taxi_fare(8, 'day'))
print(calculate_taxi_fare(15, 'day'))
print(calculate_taxi_fare(8, 'night'))
print(calculate_taxi_fare(15, 'night'))
```

Paper D — Build a Car Rental Cost Calculator

Skills: *isinstance()* · *if/elif/else* · *arithmetic* · *round()*

Function: `calculate_rental_cost(days, car_type)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_rental_cost(days, car_type):
    if not isinstance(days, int) or days <= 0:
        return "Invalid days"
    if car_type not in ("economy", "luxury"):
        return "Unsupported car type"
    if car_type == "economy":
        cost = (50 - 5) * days if days > 7 else 50 * days
    else:
        cost = 120 * days
        if days > 7:
            cost *= 1.10
    return round(cost, 2)

print(calculate_rental_cost(-2, 'economy'))
print(calculate_rental_cost(3, 'sedan'))
print(calculate_rental_cost(3, 'economy'))
print(calculate_rental_cost(10, 'economy'))
print(calculate_rental_cost(3, 'luxury'))
```

```
print(calculate_rental_cost(10, 'luxury'))
```

Paper E — Build a Repair Shop Fee Calculator

Skills: *isinstance()* · *if/elif/else* · *arithmetic* · *round()*

Function: `calculate_repair_fee(hours, service_type)`

```
print("Name: [Your Name]")
print("Class: [Your Class]")
print("Class Number: [Your Class Number]")

def calculate_repair_fee(hours, service_type):
    if not isinstance(hours, (int, float)) or hours <= 0:
        return "Invalid hours"
    if service_type not in ("basic", "specialist"):
        return "Unsupported service type"
    if service_type == "basic":
        if hours > 2:
            fee = 60 * 2 + 60 * 0.80 * (hours - 2)
        else:
            fee = 60 * hours
    else:
        fee = 100 * hours
        if hours > 2:
            fee += 30
    return round(fee, 2)

print(calculate_repair_fee(0, 'basic'))
print(calculate_repair_fee(1, 'expert'))
print(calculate_repair_fee(1.5, 'basic'))
print(calculate_repair_fee(3, 'basic'))
print(calculate_repair_fee(2, 'specialist'))
print(calculate_repair_fee(4, 'specialist'))
```